

Concept Drift Detection and Localization in Process Mining

An Integrated and Efficient Approach Enabled by Trace Clustering

Rafael Gaspar de Sousa
University of São Paulo
São Paulo, Brazil
rafael.gaspar.sousa@usp.br

Marcelo Fantinato
University of São Paulo
São Paulo, Brazil
m.fantinato@usp.br

Sarajane Marques Peres
University of São Paulo
São Paulo, Brazil
sarajane@usp.br

Hajo Alexander Reijers
Utrecht University
Utrecht, Netherlands
h.a.reijers@uu.nl

ABSTRACT

Business processes are subject to changes over time due to the need for adaptation and flexibility to a complex environment. Detecting drift as soon as possible and identifying the process elements involved, lead to a much better understanding of the process behavior, which can be a competitive edge for businesses. However, most existing approaches focus on each of these two tasks separately. Isolated approaches do not always have interfaces between them that allow you to combine solutions effectively for each corresponding task. In such cases, using the two isolated solutions together is neither feasible nor even useful from the point of view of a business analyst. This paper proposes an integrated approach to detect and locate concept drifts based on an online setting for trace clustering. Experiments with synthetic event logs with different types of control-flow changes showed that concept drifts can be detected and located efficiently.

CCS CONCEPTS

• **Applied computing** → **Business process management; Business process monitoring**; • **Information systems** → **Clustering; Data stream mining**;

KEYWORDS

Concept drift, process mining, trace clustering, data mining, business processes

ACM Reference Format:

Rafael Gaspar de Sousa, Sarajane Marques Peres, Marcelo Fantinato, and Hajo Alexander Reijers. 2021. Concept Drift Detection and Localization in Process Mining: An Integrated and Efficient Approach Enabled by Trace Clustering. In *The 36th ACM/SIGAPP Symposium on Applied Computing (SAC '21)*, March 22–26, 2021, Virtual Event, Republic of Korea. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3412841.3441918>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

SAC '21, March 22–26, 2021, Virtual Event, Republic of Korea

© 2021 Association for Computing Machinery.

ACM ISBN 978-1-4503-8104-8/21/03...\$15.00

<https://doi.org/10.1145/3412841.3441918>

1 INTRODUCTION

Traditional methods of data analysis are designed for stationary environments, where the underlying concepts do not change over time [10]. Flexibility and adaptability are concerns for organizations' managers, as such organizations face an increasing need to adapt to unusual scenarios, such as recent customer demands, competition, economics, or legislation [4]. Being able to perceive and understand changes as quickly as possible can be a competitive edge for organizations, especially when it comes to unwanted divergences that demand corrections or improvements to be spread to all personnel involved. Dynamic scenarios influence organizational operations and hence cause changes in the corresponding business processes. This situation is known as concept drift and refers to changes in the data distribution [26] common in data stream scenarios. In the organizational context, concept drift refers to a change in the behavior of the business process underlying the data.

Process mining offers a means of extracting knowledge to assist organizations managers. Process mining relies primarily on the concepts of events, cases, traces, and logs. An *event* is the occurrence of a business process activity at a given time. A *case* corresponds to a process instance and consists of events such that each event relates exactly to a case. A *trace* is a mandatory attribute of a case and corresponds to a finite sequence of events such that each event appears only once. An *event log* is a set of cases such that each event appears at most once in the entire log [24]. Although process monitoring is a potential application in this context, process mining methods and techniques usually assume that business processes are steady [21]. Literature reviews on process mining pay little attention to the proper handling of non-stationary business processes [7, 15]. To the best of our knowledge, except for the initial study on handling concept drift in process mining published in 2011 [4] and 2012 [5], interest in this topic has been developing more recently.

Concept drift-aware process mining allows for better data analysis conditions in various scenarios. Organizations supported by solutions that allows them to understand the evolution of their business processes can react quickly to adapt them when necessary. This improvement comes from the ability to split the event log into two distinct moments, i.e., before and after the concept drift. For example, process discovery can achieve more meaningful process models when concepts are segregated; otherwise, *spaghetti* process models can be generated, leading to inaccurate interpretations [23].

The main challenges for concept drift in the process mining context are the *detection* of the instant when the drift occurred, the *localization and characterization* of the drift, and the *explanation* of the process evolution [4]. The process model in Figure 1 illustrates these challenges. This model used to decide between the execution of either *B* or *C* in each process instance. After the instant t_c , both activities *B* and *C* started to occur sequentially. *Detection* should notice that a drift occurred at instant t_c . *Localization and characterization* should point out that the drift is related to the sequentialization of activities *B* and *C*. An *explanation* for this drift could be a reduction in the need for decision making.

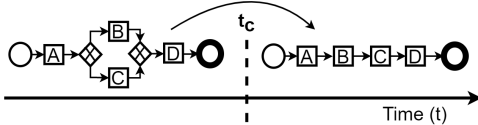


Figure 1: Example of a concept drift in a process model at instant t_c

Most approaches for concept drift in online settings focus on drift detection, providing limited information to support organizations [5, 14]. Moreover, drift localization and characterization methods are commonly based on an additional data structure (e.g., process tree) besides the one used for drift detection, thus increasing the complexity of using the approach in real-world scenarios [16, 18].

We present here an integrated approach to detect and locate concept drifts in process mining by processing an online trace stream¹. Our approach relies on trace clustering, which groups similar traces and, as a result of the clustering process, provides models (prototype vectors) that represent the process behavior. In our approach, both the detection of the drift instances and an accurate and informative description of them are produced on the same prototype vectors. Therefore, the approach allows for the efficient resolution of two tasks related to the concept drift problem.

The main contribution of this work is presenting an approach capable of *detecting* and *locating* concept drifts in an integrated way, without requiring additional data structures or process models, which simplify and streamline the procedure. The approach provides a method to monitor the evolution of trace clustering to enable drift detection. The approach also provides a method for comparing pairs of consecutive clusterings, to enable drift localization.

The remainder of this paper is organized as follows. Section 2 discusses related work on concept drift detection and localization in process mining. Section 3 details our approach, including the steps for vector representations, trace clustering, drift detection, and drift localization. Section 4 presents the evaluation and discusses the results. Finally, Section 5 concludes this paper.

2 RELATED WORK

Several approaches have been proposed for concept drift detection and localization in process mining. Detection methods are usually based on comparing traces in consecutive windows to decide whether a difference found is significant enough to be considered a drift. Localization methods are usually based on comparisons

between the process behavior before and after the drift to report differences. Each approach commonly focuses on only one of these tasks, as summarized in Table 1.

Bose et al. [4] presented one of the first methods for drift detection. Based on statistical tests, the traces are represented as feature vectors describing the activities and their occurrences. A similar approach is presented by Maaradji et al. [14] by performing the statistical test directly onto trace frequencies between two consecutive windows. Such methods present limitations concerning logs with high variability, as they hold low-frequency behaviors that compromise the reliability of the statistical test. Tackling such drawbacks, the method by Ostovar et al. [20] adds a feature extraction method to describe the process behavior as a matrix of the relationship between activities, as in the similarity-based approaches, to serve as input to the statistical tests.

Similarity-based methods for drift detection rely on processing the event log and extracting features describing the process behavior, e.g., activity frequency [5] or relationship between activities [12]. Carmona and Galvadá [5] build a reference polyhedron from the earlier data and check its intersection to more recent traces. This method does not scale well in online environments with potential multiple drifts as it needs to be restarted after finding a drift. Liu et al. [12] use process conformance techniques to check the similarity between the relationship between activities extracted from traces in a window and a reference process model discovered based on the event log. As no automatic process discovery technique can guarantee perfect discovery accuracy, spurious drifts might be detected while considering behavior not part of the actual business process.

Although the goal of the drift detection task is clear and universal over related work, there is not a common definition established concerning what information would consist the output of a drift localization approach. In general data mining, the problem of describing concept drift characteristics was structured by Webb et al. [25] through a framework that defines several quantitative features and a categorical taxonomy describing the types of drifts. However, in process mining, the concept drift localization task is interested in detailing the drift subject, i.e., identifying the aspects of control-flow changes that explain the concept drift. One work that presents an approach for both tasks is Liu et al. [12], though it aims on identifying the type of drift from a data mining perspective (sudden, incremental, or recurring). Impedovo et al. [9] focus on identifying the activities involved in the drift from the process mining perspective, as does our approach. However, their approach has the additional complexity of relying on process discovery techniques used for implementing the localization procedure. Such discovery techniques are not able to guarantee perfect accuracy, which might cause the localization of spurious differences.

From the process mining perspective, methods to compare process variants are applied in the concept drift context to describe differences in terms of business processes. Bolt et al. [3] perform statistical tests on process metrics extracted from sub-logs. However, such technique is not able to identify certain changes and presents its outputs as colored transitions systems that highlights the differences, requiring visual interpretation. Ostovar et al. [19] perform statistical tests over each relationship between activities to find the ones involved in the drift. Then, they are mapped into pre-defined change templates to be presented as output to the user.

¹Code available at github.com/rafaelgasp/trace_clustering_process_drift_detection

Table 1: Comparison of related work

Work	Drift detection			Drift localization		
	Category	Representation	Technique	Category	Representation	Output
[4]	Statistical	Activity follows/precedes	Kolmogorov–Smirnov test	-	-	-
[5]	Similarity	Activity frequency	Polyhedron overlapping	-	-	-
[14]	Statistical	Trace frequency	Chi-squared test	-	-	-
[20]	Statistical	Activities relationships	G-test	-	-	-
[12]	Similarity	Process model reference	Conformance checking	Similarity	Process model reference	Type of drift
[9]	Similarity	Graph-based	Emerging sub-graph discovery	Similarity	Process model discovery	Activities involved in drift
[19]	-	-	-	Statistical	Activities relationships	Change templates
[18]	-	-	-	Graph	Process trees	Natural language statements
[3]	-	-	-	Statistical	Transitions frequency	Visual transitions systems
[16]	-	-	-	Graph	Perspective graph	Visual matrix
This work	Similarity	Trace clustering	Clustering evolution analysis	Similarity	Trace clustering	Weighted list of entities

This method aims to identify changes in a more individual level of activities, while changes involving multiple activities and overlapping or nested changes at the same time are more challenging given the level of representation employed.

Other approaches for drift localization builds an additional graph structure to describe the process behavior. In Nguyen et al. [16], relationships between entities (e.g., activity, resource) in the event log are structured into a graph that is compared in terms of weights, common nodes, and edges. As an output, a visual matrix highlights the changes between the two process variants compared. Focusing on more complex types of process changes, Ostovar et al. [18] is based on constructing a process tree structure and finding the minimum cost sequence of edit operations that transform the process tree before the drift to the process tree after it. The result is outputted as natural language statements. Though this method works well for nested and overlapping changes with multiple activities, it does not have the same performance when localizing simpler drifts that involve individual activities in the presence of noise, as opposed to their previous work presented in Ostovar et al. [19].

Our approach is based on the use of trace clustering to analyze differences in process behavior. The analysis is quantified and allows to (i) point out the differences and (ii) analyze them from a perspective of the variation’s severity and extract useful knowledge. Most of related work focuses on either detecting or localization drifts, while our approach works on both without requiring additional data structures. This is possible as detection and localization share the information from trace clustering.

3 CONCEPT DRIFT DETECTION AND LOCALIZATION

We address concept drift in two integrated steps: (i) deciding whether the current observed process behavior reveals significant changes concerning the behavior previously presented (*drift detection*); (ii) identifying the entities involved in the differences of behavior before and after the drift (*drift localization*). Both tasks rely on properly representing the process behavior and comparing consecutive trace windows. Thus, traces are mapped into a vector representation used as input to a trace clustering algorithm. Resulting cluster information is used by the drift detection and localization methods.

Figure 2 illustrates the proposed approach applied to an online *trace stream* (the steps are numbered). Traces are processed as they arrive in the stream. First, each trace is mapped into a vector space using the chosen *vector representation* profile (1). Then, streaming *windowing* is applied based on a fixed-size window model (2). Finally, *trace clustering* is performed for that window (3), allowing to abstract and split data in an event log and hence isolate similar process behaviors. This ability is useful for dealing with concept drifts. *Drift detection* (4) is based on monitoring the evolution of clusterings over time through feature extraction. A drift is detected when there is a significant variation between two consecutive clusterings in the measured feature. When a drift is detected, *drift localization* (5) is carried out by applying the Mean Squared Error (MSE) to analyze the clusterings before and after the detected drift.

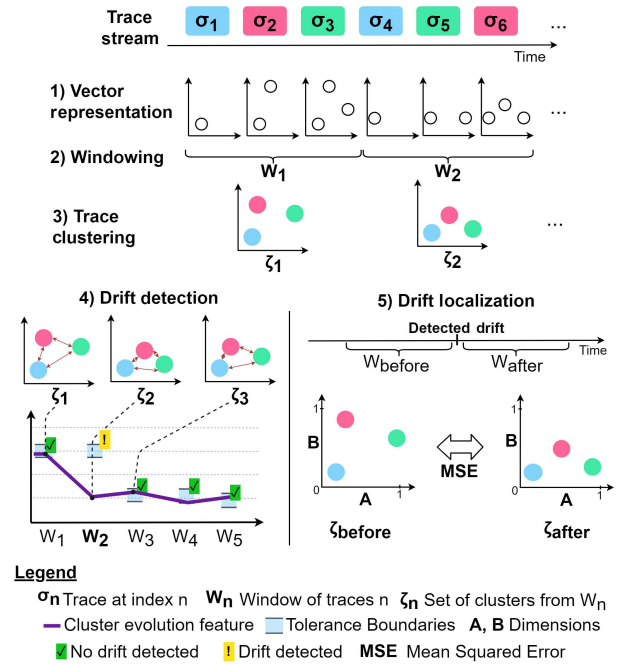


Figure 2: Overview of the proposed approach

The next sections detail the five steps of the proposed approach.

3.1 Vector representation

Data in event logs allows business process analysis from different perspectives, such as control-flow, organizational, trace, and performance [1]. These perspectives can be used to generate different representations of traces (vectors), which are called *trace profiles* [22]. The vectors of this vector space are submitted to the clustering algorithm and used to calculate trace similarity.

We use two profiles: *trace*, based on either the occurrence of activities (*Actv-Occ*) or the frequency of activities (*Actv-Frq*), and *control-flow*, based on the occurrence of transitions between activities (*Trns-Occ*). To exemplify these profiles, see Figure 3. It shows a baseline process model and the following changes from the baseline: (a) place exchange (change of order) between activities A and D; (b) loop insertion around activity A.

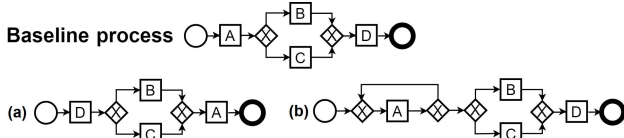


Figure 3: Examples of process model changes

Table 2 illustrates the vector representations of two traces for the *Actv-Occ* and *Trns-Occ* profiles. Occurrence is represented in a binary form. The activity profile characterizes a trace by the activities that are part of the trace. Each activity in the event log becomes a dimension in the vector representation; in this case, A, B, C, and D. The transition profile characterizes a trace by the consecutive activity pairs that are part of the trace. In this case, the dimensions in the vector representation are the pairs of activities; e.g., dimension (A:B) refers to activity A followed by activity B.

Table 2: Example of vector representations for *Actv-Occ* and *Trns-Occ*

Process		<i>Actv-Occ</i>				<i>Trns-Occ</i>							
model	Trace	A	B	C	D	A:B	A:C	B:D	C:D	D:B	D:C	B:A	C:A
Baseline	<A,B,D>	1	1	0	1	1	0	1	0	0	0	0	0
(a)	<D,B,A>	1	1	0	1	0	0	0	0	1	0	1	0

Treating concept drift requires that the vector representation incorporates features that allow identifying that the process behavior has changed. This need may be met with some limitations by different profiles. For example, according to Table 2, when comparing the two traces, the activity profile does not allow to identify the change that occurred from the baseline process model to the process model (a). The change is not identified in this case because the activity profile does not consider the order of events when representing the traces. On the other hand, also per Table 2, the transition profile allows easier discrimination between both behaviors. However, for this, the vector representation needs more dimensions.

Moreover, the *Actv-Occ* profile does not handle loops correctly, as it only considers whether the activity occurred or not. Frequency-based information can contribute to this situation. Table 3 shows the vector representations for *Actv-Occ* and *Actv-Frq* for two traces, one referring to the baseline process model and the other to the changed process model (b). Only *Actv-Frq* can distinguish the two traces, pointing out that activity A is repeatable for the second one.

Table 3: Example of vector representations for *Actv-Occ* and *Actv-Frq*

Process		<i>Actv-Occ</i>				<i>Actv-Frq</i>			
model	Trace	A	B	C	D	A	B	C	D
Baseline	<A, B, D>	1	1	0	1	1	1	0	1
(b)	<A, A, A, B, D>	1	1	0	1	3	1	0	1

3.2 Windowing and trace clustering

Once the traces are mapped into the feature vector space, they can be clustered so that similar traces are found. The key challenge here is dealing with traces in an online stream. In general data mining, online data stream clustering is usually treated with windowing. Windowing prioritizes more recent data and de-prioritizes older data [6]. Traditional clustering algorithms can be adapted to this context, using a window model, to cluster every recent data window independently. We employ the k-means++ method in our proposed approach due to its simplicity and wide application in trace clustering for other tasks in process mining.

The trace stream is processed using the landmark window model, which consists of processing batches (i.e., windows) of a fixed number of traces. The batches are identified as W_i , where i is the position of the last trace in the batch. All trace windows are individually mapped into the same feature vector space to make their variations comparable. The clustering algorithm is applied to each window so that traces representing similar process behavior are grouped. We assume that each window contains a sample of traces representative of the entire process behavior. Accordingly, the clusters are expected to be stable throughout the trace stream, as they represent an abstraction of the process behavior.

3.3 Concept drift detection

Since concept drifts occur over time, changes in clusterings need to be monitored. Oliveira and Gama [17] presented a framework capable of identifying external transitions (such as cluster merge, split, disappearance, and emergence) and internal transitions (such as cluster expansion, shrinkage, and density variations).

Different types of concept drifts can affect trace clustering in several ways, such as changing centroid positions, merging clusters, or emerging new clusters. To detail clustering evolution properties, we extract features of a *clustering*. A clustering $\zeta_i = \{C_1^i, C_2^i, \dots, C_k^i\}$ is a set of k clusters C obtained at the end of each run i of the clustering algorithm over a trace window. We extract two types of features (see Figure 4): (a) time-independent, which consider only the properties of the current clustering; and (b) time-dependent, related to the comparison of two consecutive clusterings (ζ_{i-1} is

the previous clustering; ζ_i is the current clustering). Table 4 lists the features and aggregations used here.

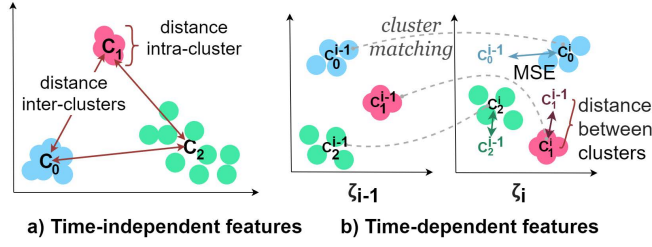


Figure 4: Examples of time-independent and time-dependent features extracted

Table 4: Clustering evolution features

Feature	Aggregations	Type
Intra-cluster distance	avg, std	Time-indep.
Inter-cluster distance	avg, min, max, sum	Time-indep.
Silhouette coefficient	-	Time-indep.
Davies-Bouldin index (DBi)	-	Time-indep.
MSE	avg, sum, count_non_zero	Time-dep.
Dist. between centroid positions	avg	Time-dep.

Our approach relies on centroid-based clustering algorithms, which represent a cluster C_j by a centroid. A centroid is the central point of a cluster and is defined here as the average of all data points that compose the cluster. Individual features are calculated for each cluster, based on its centroid, to extract time-independent features, such as intra-cluster and inter-clusters distances. For the intra-cluster distances, we consider the average pairwise distance for all data points in the cluster and for the inter-cluster distance, we apply Euclidean distance between all k centroids in a clustering ζ_i . The values obtained for all clusters are then aggregated to obtain a single value that represents the given feature for a clustering ζ_i . The validation indices Silhouette and DBi do not require aggregations as they already summarize information about all clusters at once.

Time-dependent features are based on comparing a cluster C_j^{i-1} with its matching cluster C_j^i . The choice of the matching pairs of clusters is treated as an assignment problem and solved using the Hungarian algorithm [11]. As our approach always runs clustering with a fixed k , a 1:1 matching between clusters is always obtained and hence no cluster merging or splitting occurs. MSE measures the variation of the matching centroids. For the distance between two consecutive matching clusters, the Euclidean distance measures the variation occurred in the spatial position between C_j^{i-1} and C_j^i .

Our concept drift detection method consists of verifying whether the current value of a feature has undergone a significant variation. As each feature value represents a significant group of traces, we assume they should remain stable according to the process behavior underlying the traces. The drift detection method iterates over any of the clustering evolution features over time. In each iteration, the feature value is compared to a dynamically estimated tolerance

boundary. A feature value outside the tolerance boundary represents a significant behavior variation from what was previously measured. To define what is a significant variation, we estimate upper and lower boundaries based on the average (i.e., a rolling average) and standard deviation of the feature previous values.

A detailed description of the concept drift method is shown in Algorithm 1. The method has four input parameters: v series of values of the chosen feature obtained with clustering over trace windows; rw size of the rolling average window used to analyze the behavior contained in v ; θ threshold on how many standard deviations to tolerate to consider normal behavior in v ; and mb minimum tolerated variation for a drift to be considered in v , specially useful to avoid false positives in a scenario with low standard deviation. The drift localization procedure is initiated after each detection and outputs a list of the clustering indexes where drifts were detected.

Algorithm 1 Concept drift detection algorithm

```

1: Input
2:  $v$  Vector of feature values
3:  $rw$  Rolling average window size
4:  $\theta$  Standard deviation tolerance
5:  $mb$  Minimum percentage for lower and upper boundaries
6: Output
7:  $d$  List of clustering indexes of detected drifts
8: procedure DRIFT_DETECTION( $v, rw, \theta, mb$ )
9:    $d = \emptyset$ 
10:   $buffer = \emptyset$ 
11:   $rolling\_avg = none$ 
12:   $rolling\_std = none$ 
13:   $clustering\_index = 0$ 
14:  for each  $feature\_value \in v$  do
15:    if  $size(buffer) = rw$  then
16:       $lower\_toler = \min(rolling\_avg - (rolling\_std * \theta), (1 - mb) * rolling\_avg)$ 
17:       $upper\_toler = \max(rolling\_avg + (rolling\_std * \theta), (1 + mb) * rolling\_avg)$ 
18:      if  $feature\_value < lower\_toler$  or  $feature\_value > upper\_toler$  then
19:         $d.append(clustering\_index)$ 
20:         $buffer = \emptyset$ 
21:         $drift\_localization()$ 
22:      end if
23:       $buffer.pop(0)$ 
24:    end if
25:     $buffer.append(feature\_value)$ 
26:     $rolling\_avg = average(buffer)$ 
27:     $rolling\_std = std(buffer)$ 
28:     $clustering\_index += 1$ 
29:  end for
30: end procedure

```

Algorithm 1 iterates over each value of the chosen feature in v . For each iteration, the feature value is appended to $buffer$ [ln 25]. Current $rolling_avg$ and $rolling_std$ are assigned in each iteration based on the updated $buffer$ [lns 26-27] to be used if a drift test needs to be carried out in the next iteration [lns 15-24]. When $buffer$ is full, i.e., it reached the window size rw [ln 15], a drift test is carried out. First, the lower and upper tolerance boundaries

are calculated [lns 16-17]. Then, these tolerance boundaries are used to test whether or not the current feature value should be considered a drift [ln 18]. A drift is detected if the current feature value is outside the upper and lower boundaries calculated based on *rolling_avg* and *rolling_std* from the previous iteration. If a drift is detected, d receives the clustering index of the previous feature [ln 19], *buffer* is emptied to discard old behavior [ln 20], and the localization method is triggered [ln 21], as shown in Section 3.4. When *buffer* is full, even with no drift detected, the earliest value is removed [ln 22] for appending a new feature value [ln 24].

The rolling average represents the average process behavior in a rolling average window, and the standard deviation represents the amount of variation in the rolling average window. Parameter rw controls the size of the rolling average window, affecting the smoothness in the feature values, making the detection method more sensitive to anomalies and minor changes if set to lower values. Parameter θ controls how wide the tolerance boundary is, based on the rolling standard deviation. Higher values of θ lead to wider boundaries making drift detection less frequent, while lower values of θ lead to narrow boundaries leading to more false positives. Parameter mb is used to set a minimum boundary size based on a fraction of the current rolling average that is especially effective for situations with lower rolling standard deviations that would lead to narrow boundaries.

Algorithm 1 has an inherent delay in relation to the clustering window to which the drift refers, as it demands that a certain number of traces after the drift has been processed to be able to detect the drift. For this, the algorithm [ln 18] compares the feature value with tolerance values calculated based on the rolling average and the rolling standard deviation calculated in the previous iteration.

3.4 Concept drift localization

Our concept drift localization method relies on the set of trace clusters that reflect the process behaviors generated by the online trace clustering for which drifts were detected. The drift localization method compares the clusterings before and after the detected drift to identify the specific entities involved in the control-flow change, as a drift might affect only a specific part of the business process. As drift detection runs in real time, drift localization takes advantage of the availability of clusters in the current and previous windows.

The localization method takes two clusterings as input, one immediately before and one immediately after the detected drift. The method compares the variation of the centroid positions for each pair of matching clusters from ζ_{before} and ζ_{after} . Position variation is calculated using MSE for each vector space dimension, per Equation 1, where c is the centroid of a cluster, and $c_{j,d}$ is the centroid value of cluster j in dimension d .

$$\text{MSE}(d) = \frac{1}{k} \sum_{j=1}^k \left(c_{j,d}^{\text{before}} - c_{j,d}^{\text{after}} \right)^2 \quad (1)$$

Each dimension of the vector space represents either an activity or a transition between activities (depending on the trace representation profile used). The comparison by each dimension means evaluating the difference in the process behavior (before and after) in relation specifically to each one of the activities (or transitions) of the process model. Obtaining high MSE values in a given dimension

of the centroid shows that the corresponding activity or transition behaves differently before and after the drift. The different MSE values calculated for each dimension represent the degree of change in behavior of each corresponding activity or transition.

4 EXPERIMENTS

This section presents the experiments conducted to evaluate the proposed approach.

4.1 Setup and execution of trace clusterings

We conducted experiments on synthetic event logs with simulated concept drifts. We used the event logs from the *Business Process Drift* collection [13, 14], available in *4TU.Centre for Research Data*. The logs have different types of concept drifts, referring to changes in the process control-flow in the Loan Application baseline process model. This process model has 15 activities, one start and three end events, loops, parallelism, and alternative branches (cf. Figure 5).

Table 5 lists the changes entered in the log. The baseline process model was changed using 12 patterns of changes, categorized as: (I) Insertion, (R) Resequentialization, and (O) Optionalization. For each change pattern, three event logs were generated, with 5,000, 7,500, and 10,000 traces. Each event log was generated with nine concept drifts, i.e., for each 10% of the log size, the baseline process model traces were alternated with the changed process model traces.

Table 5: Change patterns present in event log and support by trace representation profile of our approach

Change pattern	Description	Category	Actv-Occ	Actv-Frq	Trns-Occ
cb	Make fragment skippable/non-skippable	O	x	x	x
cd	Synchronize two fragments	R			x
cf	Make two fragments conditional/sequential	R	x	x	x
cm	Move fragment into/out of conditional branch	I	x	x	x
cp	Duplicate fragment	I		x	x
fr	Change branching frequency	O	x	x	x
lp	Make fragment loopable/non-loopable	O		x	x
pl	Make two fragments parallel/sequential	R			x
pm	Move fragment into/out of parallel branch	I	x	x	x
re	Add/remove fragment	I	x	x	x
rp	Substitute fragment	I	x	x	x
sw	Swap two fragments	I			x

Table 5 also shows which of the three profiles is able to represent each of the 12 patterns of change (marked with an x signal). The *Trns-Occ* profile can represent all the change patterns. The same does not apply to *Actv-Occ* and *Actv-Frq*. For example, since *Actv-Occ* can identify only whether or not an activity is present in the trace, it cannot represent information on the existence or not of loops (see pattern *lp*). On the other hand, *Actv-Frq* can identify how many times an activity is present in the trace, and thus it can represent information on the existence or not of loops.

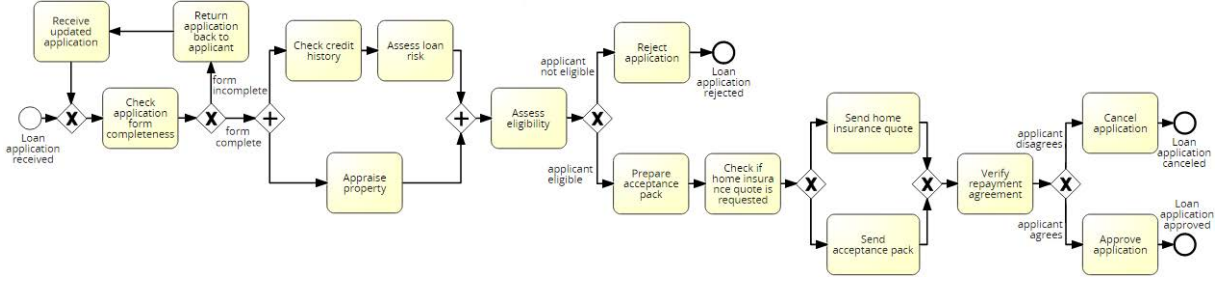


Figure 5: Loan applications baseline process model [8, 13]

We run trace clustering with *k-means++* algorithm. The vector space representation was built using prior knowledge of all possible activities in the business process. Table 6 lists the values used for the parameters related to windowing and trace clustering.

Table 6: Trace clustering parameters

Parameter	Values
Profile	<i>Actv-Occ</i> , <i>Actv-Frq</i> , <i>Trns-Occ</i>
Windowing	Landmark
Window size	50, 75, 100, 125, 150, 200, 250
Algorithm	<i>k-means++</i>
k	2, 3, 6

4.2 Detecting concept drifts

We run the drift detection method (see Algorithm 1) for the clustering evolution features in Table 4. Table 7 lists the values used for the parameters of this algorithm.

Table 7: Drift detection parameters

Parameter	Values	Parameter	Values
<i>Feature</i>	12 options in Table 4	<i>rw</i>	3, 4
<i>mb</i>	0.3%, 0.5%, 0.7%, 1.0%	θ	1.0, 1.5, 2.0

Figure 6 illustrates the results of the calculations performed by Algorithm 1, for the event log with 10,000 traces and the change pattern *cb*, using *wind_size* = 100, *feature* = *avg_dist_between_centroid_positions*, *k* = 2, *rw* = 3, θ = 2, and *mb* = 1.0%. The red line represents the aggregated value of the chosen feature measured for each window, i.e., for each clustering. Significant changes in feature values would indicate the presence of concept drifts. The red line suffers minor variations, which should not indicate drifts, and major variations, which should indicate drifts. The rolling average (green line) is calculated on the feature values taking a given number of last windows, in this case, the last three windows, and it is used to avoid false positives detections. As result, the rolling average line smooths the data to the feature values. The blue zones represent the minimum and maximum limits for a feature value change not to be considered significant, i.e., the tolerance boundary.

A drift is detected when the feature value lies outside the tolerance boundary. The tolerance boundary has a dynamic behavior

(increasing and decreasing) based on the variation presented by the rolling average and its standard deviation. With a constant green line, the blue zone is very narrow; on the other hand, when the green line varies, the blue zone is relaxed, otherwise, small variations in the feature values would be falsely detected as drifts. For example, near trace 7,500 a false drift could have been detected. In this example, all concept drifts were correctly detected with exactly one delay window, which is indicated by vertical lines.

The experiments were performed considering all combinations of trace clustering parameters (63 combinations, cf. Table 6) combined with all combinations of drift detection parameters (288 combinations, cf. Table 7). These parameter combinations were applied to the three event logs (with 5,000, 7,500 and 10,000 traces) combined with the 12 change patterns (cf. Table 5). Drift detection results were evaluated using *F-Score* to capture the trade-off between precision and recall and *mean delay* to measure the average delay required (in the number of windows) for a drift to be detected. Table 8 shows the best results achieved for drift detection, considering the parameter combinations for each representation profile. A ranking considering a cost-benefit ratio in terms of which best solves the largest set of change patterns was used to select the best parameter combinations. The highest F-Score values for each change pattern (provided they are greater than or equal to 0.9) are pointed in bold. Change patterns not supported by a given profile were disregarded.

According to Table 8, one can observe that for both activity profiles, the *Actv-Frq* profile presented F-Score results that were always the same or better than *Actv-Occ*. Thus, assuming that these best F-Score results represent the general behavior of the proposed drift detection method, *Actv-Frq* could always be chosen instead of *Actv-Occ*. The *Actv-Frq* profile also showed some better F-Score results than *Trns-Occ*, but the latter supports some change patterns not supported by the former. Thus, we can understand that *Trns-Occ* is good for supporting exclusive change patterns while *Actv-Frq* is good for presenting in some cases better F-Score results than *Trns-Occ*. Table 9 lists the trace clustering and drift detection parameters that led to the best results presented in Table 8.

Drifts detected with over two window delays were considered false positives, as they were detected far from where the drift occurred. These drifts were probably detected for reasons other than those expected in the experiment. As a result, all mean delays in Table 8 are below 2.0. The transition profile showed generally lower mean delays than activity profiles but at the overall cost of getting worse F-Score results when compared with the *Actv-Frq* profile.

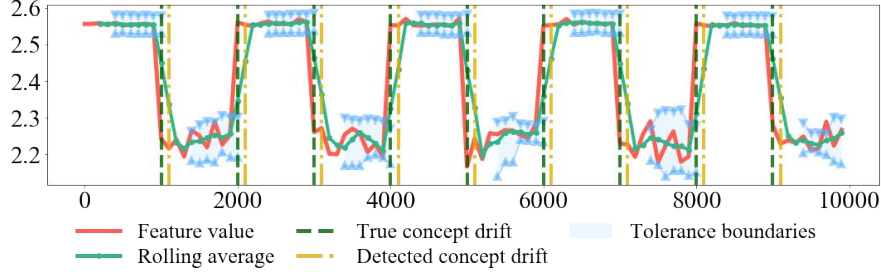


Figure 6: Execution of the drift detection algorithm for the log with 10,000 traces and the change pattern *cb*, using $wind_size = 100$, $k = 2$, $feature = avg_dist_between_centroid_positions$, $rw = 3$, $\theta = 2$, and $mb = 1.0\%$

Table 8: Best results achieved for drift detection, considering the parameter combinations with the best cost-benefit ratio for each representation profile

Change pattern	Actv-Occ		Actv-Frq		Trns-Occ	
	F-Score	Mean delay	F-Score	Mean delay	F-Score	Mean delay
cb	0.98	1.63	0.98	1.63	0.76	1.25
cd	-	-	-	-	0.90	1.25
cf	0.98	1.70	0.98	1.70	0.98	1.07
cm	0.95	1.63	0.95	1.63	0.91	1.05
cp	-	-	0.96	0.96	1.00	0.96
fr	0.05	0.66	0.06	0.66	0.12	1.33
lp	-	-	1.00	1.59	0.76	0.99
pl	-	-	-	-	0.90	1.70
pm	0.98	1.63	0.98	1.63	1.00	1.00
re	0.88	1.00	1.00	1.00	0.72	1.00
rp	0.94	1.00	0.94	1.00	0.98	1.00
sw	-	-	-	-	1.00	1.00

Table 9: Parameters used to obtain the best average drift detection results shown in Table 8

Trace representation profile	Feature	Clustering parameters		Detection parameters		
		$wind_size$	k	rw	θ	mb
Actv-Occ	MSE_count_non_zero	125	2	3	1	1.0%
Actv-Frq	MSE_count_non_zero	125	2	3	1	0.3%
Trns-Occ	MSE_count_non_zero	125	2	3	2	0.3%

Still per Table 8, *fr* had very poor F-Score values. In fact, this change pattern has a slight impact on the process behavior as it only changes the frequency of two activities associated with alternative paths in the process models (related to a XOR gateway). We verified that our trace clustering-based method is not very sensitive to this type of change. However, some particular parameter combinations showed better results specifically for *fr*, while showing poor results for other change patterns. For example, *fr* achieved F-Score of 0.91 with mean delay of 1.21 with these parameters: *profile = activity/frequency*, *feature = avg_dist_between_centroids_positions*,

$wind_size = 150$, $k = 2$, $rw = 3$, $\theta = 2$, and $mb = 0.7\%$. This phenomenon also occurs for the other change patterns, i.e., each one of them can individually present F-Score results better than those shown in Table 8 when taking particular parameter combinations.

We compared our F-Score results with the ones presented by Maaradji et al. [14], as it is the only one evaluated with the same event logs and conditions as ours. Table 10 lists their results as a baseline compared with the results achieved by our concept drift detection method. Compared to our most complete best results, i.e., the *Trns-Occ* profile (cf. Table 8), our method performed better or equivalent for half of the cases. Moreover, when considering the best results by each change pattern individually, taking the best parameter combination for each, our method performed better or equivalent for all change patterns. This shows that better results can be obtained with our method by adjusting the parameters according to the change pattern under analysis. Anyway, our main goal is to present an integrated approach that also supports the localization of the detected drifts, as discussed in the following.

Table 10: Comparison between the baseline [14] and our concept drift detection method (F-Score results)

Change pattern	Baseline	Our drift detection method	
		<i>Trns-Occ</i> (cf. Table 8)	Best results by change pattern individually
cb	0.92	0.76	1.00
cd	0.88	0.90	0.96
cf	0.98	0.98	1.00
cm	1.00	0.91	1.00
cp	1.00	1.00	1.00
fr	0.75	0.12	0.91
lp	1.00	0.76	1.00
pl	1.00	0.90	1.00
pm	1.00	1.00	1.00
re	1.00	0.72	1.00
rp	0.96	0.98	0.98
sw	1.00	1.00	1.00

4.3 Locating concept drifts

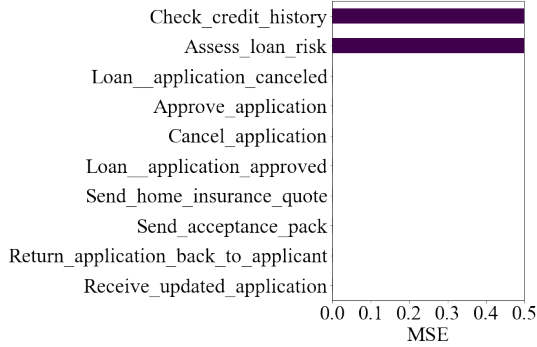
To experiment the second method of our approach, we assumed that the first method worked perfectly. As we had information a

priori (our ground truths), we simulated a perfect execution of the drift detection method to specifically evaluate the drift localization method. The goal here was to evaluate whether the method would be able to correctly identify the elements of the process (activities or transitions) involved in each detected concept drift.

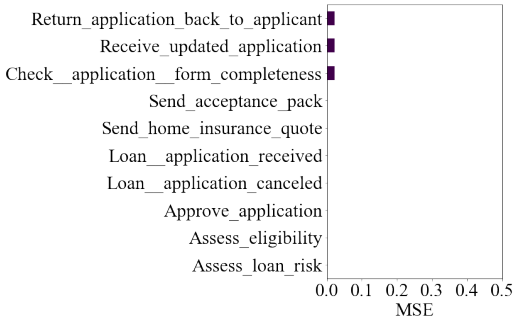
To better illustrate the localization method, Figure 7 shows the MSE for the top 10 dimensions on false-positive and true-positive drift detection for the event log *cp*. The activities expected to be involved in this drift are *Check credit history* and *Assess loan risk*. Both of these activities hold the highest values for MSE when comparing the clustering situations before and after a true-positive drift detection, as shown in Figure 7a. When a detected false-positive drift is submitted to the localization method, entities not related to an expected drift score higher in the MSE, as shown in Figure 7b. This can provide evidence when analyzing subtle changes in the process behavior that led to false-positive detections.

Figure 7: Drift localization results for the 10,000 traces *cp* event log with *Actv - Frq* representation, $k = 2$ and $window_size = 125$

(a) Drift localization of the true-positive drift detected at index 7,000.



(b) Drift localization of the false-positive drift detected at index 6,750.



Overall results were evaluated using *Area Under the ROC Curve* (*AUC*) to capture the trade-off between true positive and false positive rates and analyze how MSE can distinguish the elements involved in the drift. Table 11 shows the best results achieved for drift localization, considering the parameter combinations for each representation profile. A ranking considering a cost-benefit ratio in

terms of which best solves the largest set of change patterns was used to select the best parameter combinations. The highest *AUC* values for each change pattern are highlighted in bold. Change patterns not supported by a given profile were disregarded.

Table 11: Best results achieved for drift localization, considering the parameter combinations with the best cost-benefit ratio for each representation profile

Change pattern	Actv-Occ	Actv-Frq	Trns-Occ	Change pattern	Actv-Occ	Actv-Frq	Trns-Frq
cb	1.00	1.00	0.69	lp	-	0.99	0.80
cd	-	-	0.83	pl	-	-	0.92
cf	1.00	1.00	0.98	pm	1.00	1.00	0.95
cm	0.99	0.99	1.00	re	1.00	0.99	0.99
cp	-	1.00	0.97	rp	1.00	1.00	1.00
fr	1.00	1.00	1.00	sw	-	-	0.93

The *Actv-Occ* and *Actv-Frq* profiles performed very well for all the change patterns they support, always reaching at least 0.99. *Trns-Occ* performed well in general, although not as good as the activity-based profiles. As *Trns-Occ* has higher dimensionality, the centroids are more naturally subject to fluctuations in the dimensions that negatively affect its effectiveness. To address this issue, minor variations could be disregarded or natural fluctuations could be removed when calculating MSE.

The clustering parameters used for each case are: *Actv-Occ* ($k = 2$, $window_size = 100$), *Actv-Frq* ($k = 2$, $window_size = 100$), *Trns-Occ* ($k = 2$, $window_size = 250$). *Trns-Occ* performed better with a larger window size probably due to its greater dimensionality.

Still in Table 11, one can see that *fr* achieved good results like the other change pattern for the same parameter combination. This is a different result than shown in Table 8 for the detection of drifts. *fr* obtained good results for the localization, despite its inherent characteristics already discussed, because even small variations (in the MSE calculation) can be significant for the drift localization method, which does not apply for the drift detection method. If the detection method parameters were set to detect *fr*, it would become very sensitive, and hence many false positives would be detected.

5 CONCLUSION

In this paper, we proposed an integrated concept drift detection and localization approach based on online trace clustering. The experiments with synthetic event logs with different types of concept drifts presented encouraging results showing that concept drifts are captured by trace clustering. Although our approach has not outperformed the baseline for drift detection in all cases, we found the potential for extensions and parameter calibration. Our drift localization method was able to identify the entities involved in the concept drift in most cases with no errors.

To ensure that the clusterings are bound to the same vector space over time, we assumed prior knowledge of all possible activities and transitions in the event log. In addition, our approach is capable of dealing only with drifts representable through trace clustering. Thus, the approach is subject to the limitations of the trace vector representation and clustering algorithms.

Avenues for expanding this research include: (i) exploring more complex streaming clustering algorithms with dynamic number of clusters and different windowing models, such as an adaptive windowing model, as discussed by Bifet and Gavaldá [2] and Maaradji et al. [14]; (ii) exploring more trace representation profiles, such as transition frequency, and combining multiple profiles to enhance trace clustering; (iii) ensembling multiple features when monitoring the trace clustering evolution for drift detection; (iv) exploring the detection and localization of composite change patterns; and (v) carrying out experiments with more complex processes and real-world event logs.

6 ACKNOWLEDGMENTS

The authors thank Universidade de São Paulo and Amazon Web Services (AWS) for the support of this research in Cloud Manufacturing Services, in the scope of USP-AWS collaboration; the São Paulo Research Foundation – FAPESP-Brazil – process numbers 2017/26487-4 and 2017/26491-1.

REFERENCES

- [1] Annalisa Appice and Donato Malerba. 2016. A Co-Training Strategy for Multiple View Clustering in Process Mining. *IEEE Trans. Serv. Comput.* 9, 6 (2016), 832–845.
- [2] Albert Bifet and Ricard Gavaldá. 2007. Learning from Time-Changing Data with Adaptive Windowing. *Proceedings of the 7th SIAM International Conference on Data Mining* 7.
- [3] Alfredo Bolt, Massimiliano de Leoni, and Wil van der Aalst. 2018. Process variant comparison: Using event logs to detect differences in behavior and business rules. *Inf. Syst.* 74 (2018), 53 – 66.
- [4] R. P. Jagadeesh Chandra Bose, Wil van der Aalst, Indre Žliobaitė, and Mykola Pechenizkiy. 2011. Handling Concept Drift in Process Mining. In *23rd Int. Conf. on Adv. Inf. Syst. Eng.* 391–405.
- [5] Josep Carmona and Ricard Gavaldá. 2012. Online Techniques for Dealing with Concept Drift in Process Mining. In *11th Int. Symp. Adv. in Intel. Data Anal.* 90–102.
- [6] Matthias Carnein and Heike Trautmann. 2019. Optimizing Data Stream Representation: An Extensive Survey on Stream Clustering Algorithms. *Bus. & Inf. Syst. Eng.* 61 (2019), 277–297.
- [7] Cleiton dos S. Garcia, Alex Meincheim, Elio R. Faria Junior, Marcelo R. Dallagassa, Denise Maria V. Sato, Deborah R. Carvalho, Eduardo A. P. Santos, and Edson Emilio Scalabrin. 2019. Process mining techniques and applications – A systematic mapping study. *Expert Syst. Appl.* 133 (2019), 260 – 295.
- [8] Marlon Dumas, Marcello La Rosa, Jan Mendling, and Hajo A. Reijers. 2013. *Fundamentals of Business Process Management*.
- [9] Angelo Impedovo, Paolo Mignone, Corrado Loglisci, and Michelangelo Ceci. 2020. Simultaneous Process Drift Detection and Characterization with Pattern-Based Change Detectors. In *Discovery Science*. Springer International Publishing, Cham, 451–467.
- [10] Paulo M. Gonçalves Jr., Silas G.T. de Carvalho Santos, Roberto S.M. Barros, and Davi C.L. Vieira. 2014. A comparative study on concept drift detectors. *Expert Syst. Appl.* 41, 18 (2014), 8144 – 8156.
- [11] Harold William Kuhn. 1955. The Hungarian method for the assignment problem. *Naval Research Logistics Quarterly* 2, 1-2 (1955), 83–97.
- [12] Na Liu, Jiwei Huang, and Lizhen Cui. 2018. A Framework for Online Process Concept Drift Detection from Event Streams. In *IEEE Int. Conf. on Serv. Comput.* 105–112.
- [13] Abderrahmane Maaradji, Marlon Dumas, Marcello La Rosa, and Alireza Ostovar. 2015. Business Process Drift. Dataset, Queensland Univ. of Technology, Australia.
- [14] Abderrahmane Maaradji, Marlon Dumas, Marcello La Rosa, and Alireza Ostovar. 2015. Fast and Accurate Business Process Drift Detection. In *Bus. Process Manage.* 406–422.
- [15] Ana Rocío C. Maita, Lucas C. Martins, Carlos Ramón L. Paz, Laura Rafferty, Patrick C. K. Hung, Sarajane M. Peres, and Marcelo Fantinato. 2017. A systematic mapping study of process mining. *Enterprise Inf. Sys.* (2017).
- [16] Hoang Nguyen, Marlon Dumas, Marcello La Rosa, and Arthur H. M. ter Hofstede. 2018. Multi-perspective Comparison of Business Process Variants Based on Event Logs. In *Conceptual Modeling*. 449–459.
- [17] Márcia Oliveira and João Gama. 2010. MEC - Monitoring Clusters’ Transitions. In *5th Starting AI Researchers’ Symp.* 212–224.
- [18] Alireza Ostovar, Sander J. J. Leemans, and Marcello La Rosa. 2020. Robust Drift Characterization from Event Streams of Business Processes. *ACM Trans. Knowl. Discov. Data* 14, 3, Article 30 (2020), 57 pages.
- [19] Alireza Ostovar, Abderrahmane Maaradji, Marcello La Rosa, and Arthur ter Hofstede. 2017. Characterizing drift from event streams of business processes. In *29th Int. Conf. on Adv. Inf. Syst. Eng.* 210–228.
- [20] Alireza Ostovar, Abderrahmane Maaradji, Marcello La Rosa, Arthur H.M. ter Hofstede, and Boudewijn van Dongen. 2016. Detecting drift from event streams of unpredictable business processes. In *35th Int. Conf. on Conceptual Model.*
- [21] Alexander Seeliger, Timo Nolle, and Max Mühlhäuser. 2017. Detecting Concept Drift in Processes Using Graph Metrics on Process Graphs. In *9th Conf. on Subject-Oriented Bus. Process Manage.* Article 6, 10 pages.
- [22] Minseok Song, Christian W. Günther, and Wil van der Aalst. 2008. Trace clustering in process mining. In *4th Workshop on Bus. Process Intell.* 51–62.
- [23] Wil van der Aalst. 2011. *Process Mining: Discovery, Conformance and Enhancement of Business Processes* (1st ed.). 352 pages.
- [24] Wil van der Aalst. 2016. *Process Mining: Data Science in Action* (2nd ed.).
- [25] Geoffrey I. Webb, Roy Hyde, Hong Cao, Hai L. Nguyen, and Francois Petitjean. 2016. Characterizing Concept Drift. *Data Min. Knowl. Discov.* 30, 4 (2016), 964–994.
- [26] Indre Žliobaitė, Mykola Pechenizkiy, and João Gama. 2016. An Overview of Concept Drift Applications. In *Big Data Analysis*. Vol. 16. 91–114.